CSI605: OBJECT-ORIENTED SOFTWARE ENGINEERING

FACULTY OF NATURAL AND APPLIED SCIENCES

SCHOOL OF COMPUTER, MATHEMATICS & STATISTICS SCIENCE



UNIVERSITY PUBLICATION MANAGEMENT SYSTEM

OVERVIEW OF IMPLEMENTATION REPOSITORY

Lecturer: Prof Ayalew

GROUP MEMBERS:

1. Keletso Joy Ditlhotlhole-202002023 **(Team Leader)**
2. Kgosi Philip Zilwa-202005452
3. Phazha Chakalisa-202106101
4. Kagiso Mpopang 202607142
5. Archibald Mareme-201904960

Due Date: 06/11/2026

TABLE OF CONTENTS

1.TEAM RESPONSIBILITIES 3

2. IMPLEMENTATION AND FILE STRUCTURE 4

2.1 Simplified File Structure 4

2.2 Angular Frontend 5

2.3 Node.js API and Backend 5

2.4 Configuration Files 5

2.5 Controllers 6

2.6 Middleware 6

2.7 Routes 6

2.8 Keycloak Authentication 6

2.9 Notification Service 6

2.10 Database and Supporting Files 6

2.11 Files Excluded from the Documentation 7

2.12 Request Flow 7

3. OVERVIEW OF UI 8

1.TEAM RESPONSIBILITIES

- **Keletso Joy Ditlhotlhole – Group Leader / Frontend Development**
  Served as the group leader, coordinating team activities and ensuring an organized development process, including maintaining order within the shared repository. She contributed to UI design and frontend implementation and was specifically responsible for preparing the documentation and repository structure summary for the project documentation.
- **Kgosi Zilwa – Authentication and Access Control**
  Primarily responsible for implementing authentication and role-based access control, ensuring that users had the appropriate permissions to access system pages and core functionality, particularly administrator features and restricted pages.
- **Phazha Chakalisa – Backend Development**
  Primarily contributed to the development and implementation of backend functionality supporting the system's core features.
- **Archibald Mareme – Backend Development**
  Primarily contributed to backend development and integration**,** ensuring that server-side functionality supported the required system operations.
- **Kagiso Mpopang – Backend Development**
  Primarily contributed to the implementation and integration of backend functionality required by the system.
- **Entire Team – Documentation, Backend, Integration and Maintenance**
  The entire team collaboratively contributed to project documentation, backend development, system integration and maintenance. Team members participated in testing, troubleshooting, resolving implementation issues, refining functionality, and ensuring that the frontend, backend, authentication, database and other system components worked together correctly.

2. IMPLEMENTATION AND FILE STRUCTURE

The project follows a client-server structure. The Angular frontend provides the user interface, while the Node.js backend processes requests, applies business rules, communicates with the database and supports file uploads and email notifications. Keycloak provides authentication and role-based access control.

2.1 Simplified File Structure

```
PROJECT ROOT/
|
|-- api/                        # Additional Node.js API service
|   |-- server.js                # Starts the API service
|   `-- package.json             # API dependencies and scripts
|
|-- backend/                     # Main Node.js backend
|   |-- src/
|   |   |-- config/
|   |   |   |-- database.js      # Database connection configuration
|   |   |   `-- keycloak.js      # Keycloak integration configuration
|   |   |-- controllers/
|   |   |   `-- publication.controller.js  # Publication request handling
|   |   |-- middleware/
|   |   |   |-- auth.middleware.js   # Verifies authentication tokens
|   |   |   |-- role.middleware.js   # Enforces user roles
|   |   |   `-- upload.middleware.js # Handles uploaded files
|   |   |-- routes/
|   |   |   |-- auth.routes.js       # Authentication endpoints
|   |   |   `-- publication.routes.js # Publication endpoints
|   |   |-- services/
|   |   |   `-- email.service.js     # Sends email notifications
|   |   `-- server.js                # Starts the backend server
|   |-- .env                         # Backend environment variables
|   `-- package.json                 # Backend dependencies and scripts
|
|-- keycloak/
|   `-- realm-export.json            # Realm, clients, roles and users configuration
|
|-- src/                         # Angular frontend
|   |-- app/
|   |   |-- models/
|   |   |   `-- publication.model.ts # Publication data structure
|   |   |-- Publication Management Subsystem/
|   |   |   |-- review-publication/  # Lists publications for review
|   |   |   |-- review-publication-detail/ # Shows review details
|   |   |   |-- Submit Publication/  # Publication submission feature
|   |   |   `-- view-search-publication/ # Search and view feature
|   |   |-- services/
|   |   |   |-- auth.service.ts      # Frontend authentication operations
```

```
|  |  |  |-- auth.interceptor.ts  # Adds tokens to HTTP requests
|  |  |  |-- publication.service.ts # Calls publication API
|  |  |  `-- role.guard.ts        # Protects routes by role
|  |  |-- User Interface Subsystem/
|  |  |  |-- Dashboard/           # General and student dashboards
|  |  |  `-- Login/               # Login page and logic
|  |  |-- not-found/              # Invalid-route page
|  |  |-- app.component.ts        # Root component logic
|  |  |-- app.component.html      # Root component template
|  |  |-- app.config.ts           # Application-wide providers
|  |  `-- app.routes.ts           # Main frontend routes
|  |-- assets/images/             # Static images and logo
|  |-- index.html                 # Page hosting the Angular application
|  |-- main.ts                    # Angular entry point
|  `-- styles.css                 # Global styles
|
|-- angular.json                  # Angular build configuration
|-- package.json                  # Main project dependencies and scripts
|-- tsconfig.json                 # TypeScript compiler configuration
|-- docker-compose.yml            # Container service configuration
|-- database.sql / db.sql         # Database scripts
`-- Publications_API.postman_collection.json # API test collection
```

2.2 Angular Frontend

The src folder contains the Angular user interface. It controls what users see and how they interact with the system.

- Components combine TypeScript logic, HTML templates and CSS or SCSS styling.
- The publication management components support submission, searching, viewing and reviewing publications.
- Angular services communicate with the backend API and manage shared functions such as authentication.
- app.routes.ts maps browser paths to components, while role.guard.ts prevents unauthorised navigation.

2.3 Node.js API and Backend

The api and backend folders contain server-side JavaScript. The backend receives HTTP requests from Angular, processes them and returns responses.

- server.js creates and starts the Node.js server and registers middleware and routes.
- package.json records backend libraries and commands for starting or developing the server.
- .env stores environment-specific values such as ports, database settings and authentication configuration. It should not be committed with real secrets.

2.4 Configuration Files

The backend/src/config folder centralises external system settings.

- database.js establishes and manages the connection to the project database.
- keycloak.js configures communication between the backend and Keycloak.

2.5 Controllers

Controllers contain the main request-handling logic. publication.controller.js receives publication requests from the routes, validates or processes the supplied data, calls the required database operations and sends an HTTP response to the frontend.

2.6 Middleware

Middleware runs between the incoming request and the final controller.

- auth.middleware.js verifies that the request contains a valid authentication token.
- role.middleware.js checks whether the authenticated user has the required role or permission.
- upload.middleware.js processes upload publication documents before the controller handles them.

2.7 Routes

Routes define the backend endpoints that the Angular frontend can call.

- auth.routes.js provides authentication-related endpoints.
- publication.routes.js provides endpoints for submitting, retrieving, searching, reviewing or updating publications.
- Routes forward valid requests to middleware and then to the appropriate controller.

2.8 Keycloak Authentication

The keycloak folder contains realm-export.json. This file stores the exported Keycloak realm configuration, including clients, roles and authentication settings. Angular obtains and sends access tokens, the backend validates them, and role checks determine which functions a user may access.

2.9 Notification Service

The backend/src/services/email.service.js file provides the notification functionality identified in the repository. It is used to send email notifications about relevant publication events, for example submission or review status changes. The controller can call this service after the relevant backend action succeeds.

2.10 Database and Supporting Files

Database scripts define or reproduce the project database, while supporting configuration files help run and test the system.

- database.sql, db.sql and iteration2dbdump.sql contain database definitions or saved database data.
- docker-compose.yml defines the services that can run together in containers, such as the application, database and Keycloak.

- Publications_API.postman_collection.json contains saved requests for testing the API in Postman.
- angular.json and tsconfig.json control Angular building and TypeScript compilation.

2.11 Files Excluded from the Documentation

The expanded node_modules directories, generated Angular cache files, uploaded documents, lock-file internals and individual test files are excluded from the simplified structure. They are not primary source-code modules and would make the documentation unnecessarily long.

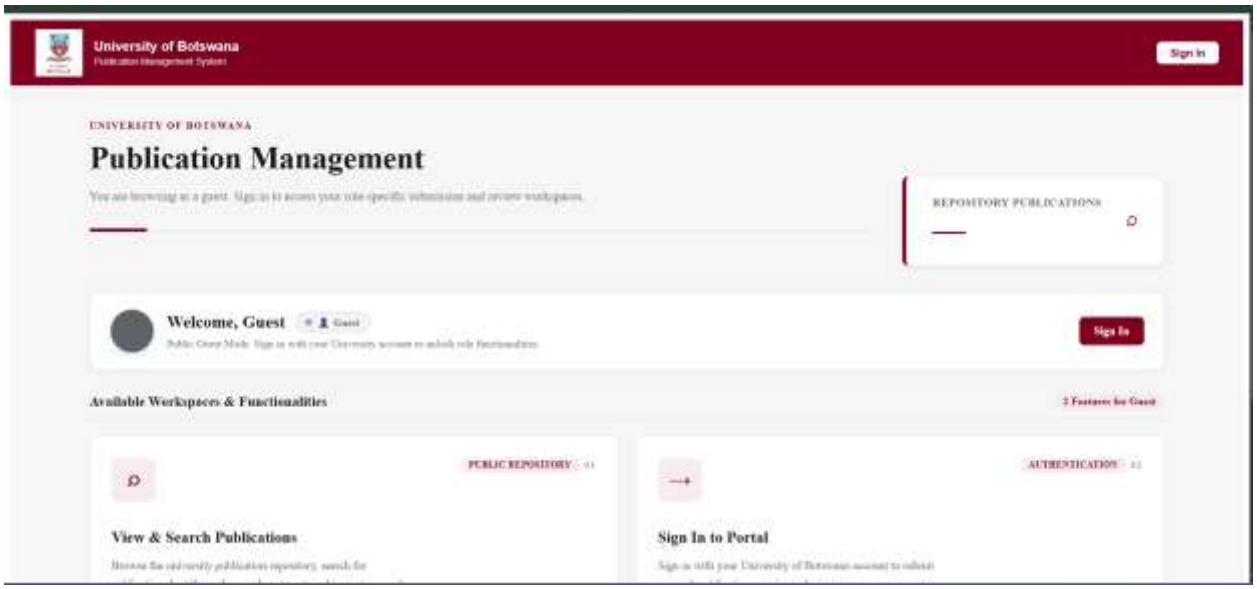
2.12 Request Flow

A typical request flows through the system as follows:

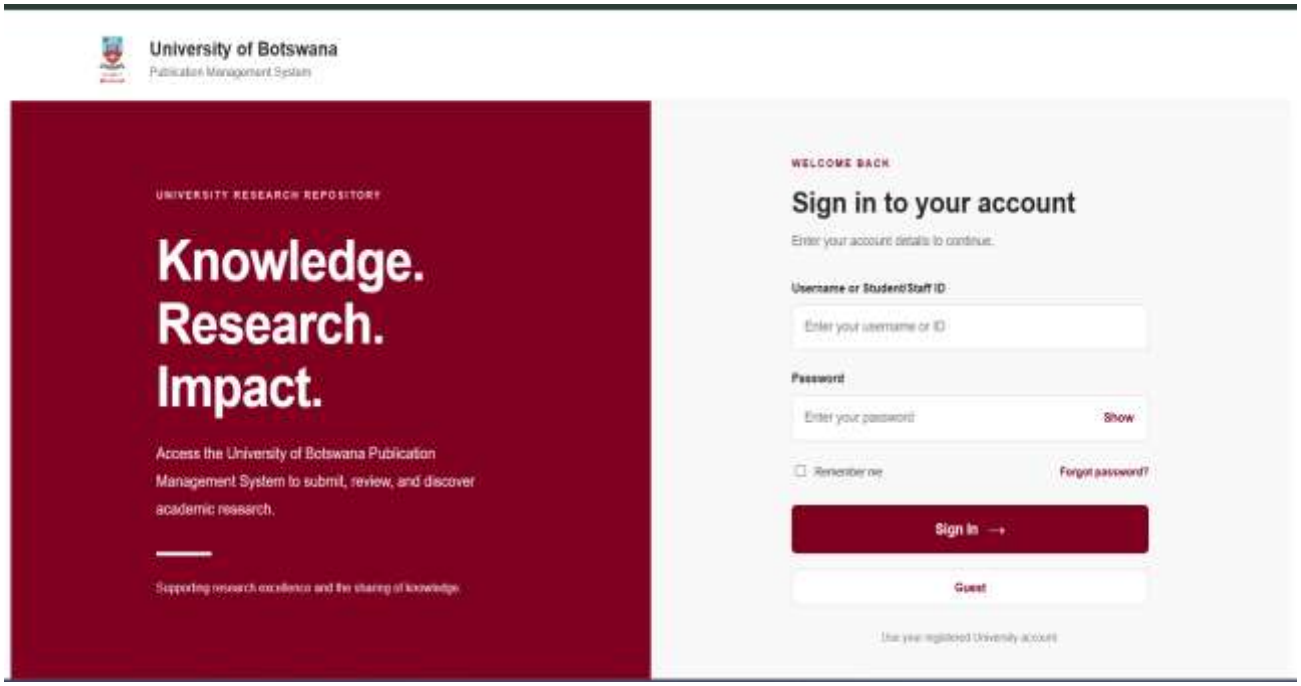
1. The user performs an action in an Angular component.
2. An Angular service sends an HTTP request to a backend route.
3. Authentication and role middleware validate the request.
4. The route forwards the request to the publication controller.
5. The controller performs database or file-processing operations.
6. The email service sends a notification when required.
7. The backend returns a response and Angular updates the interface.

3. OVERVIEW OF UI

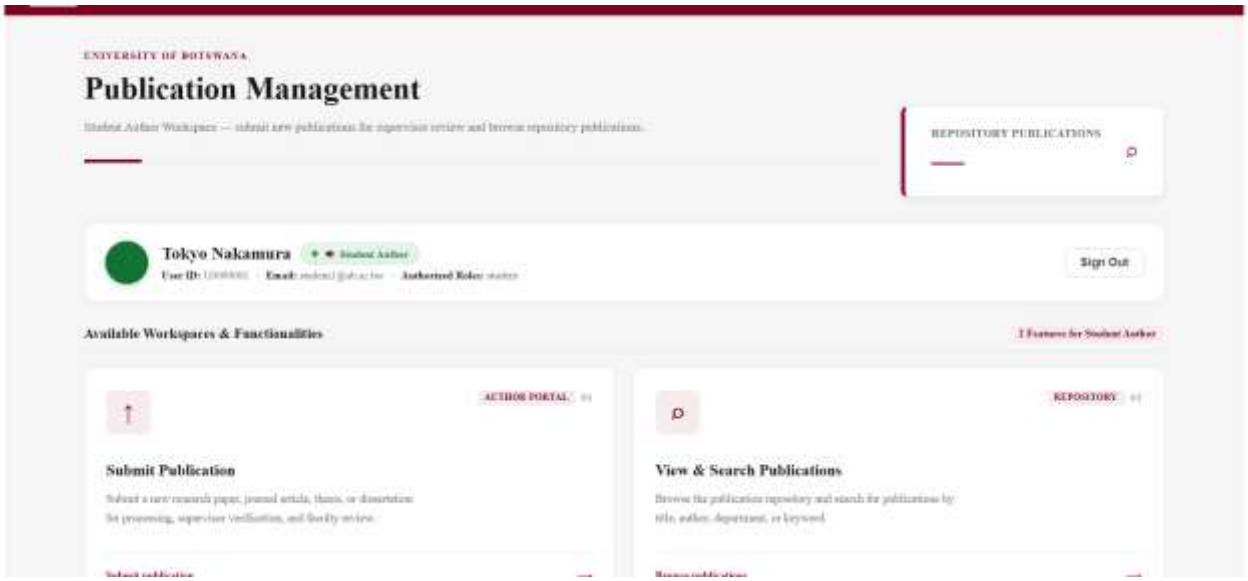
Guest Dashboard



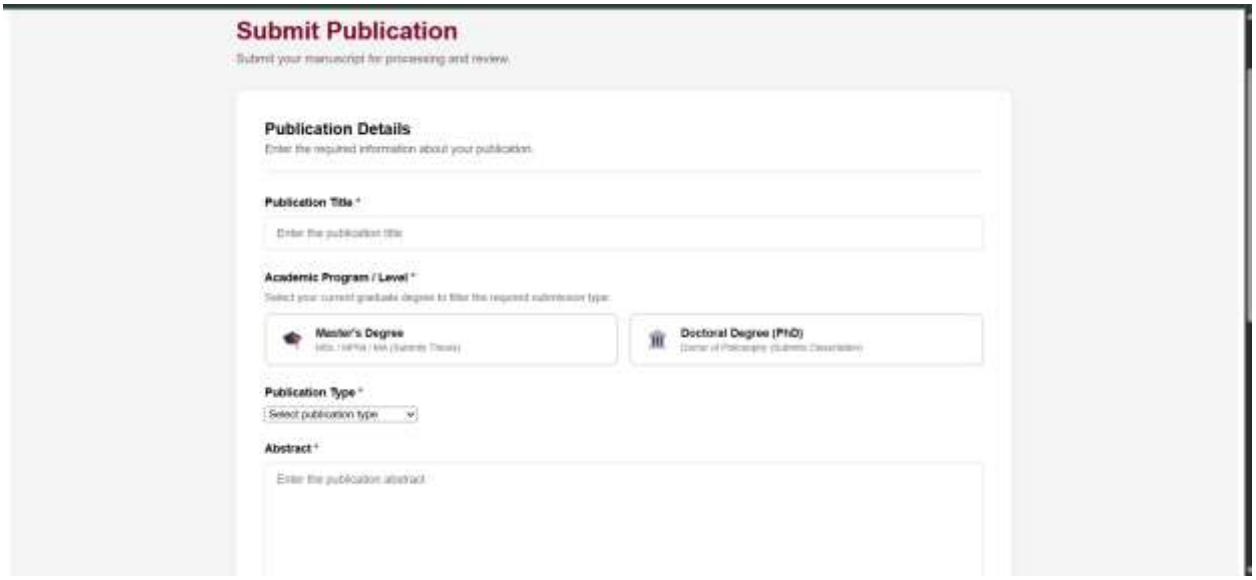
Login page

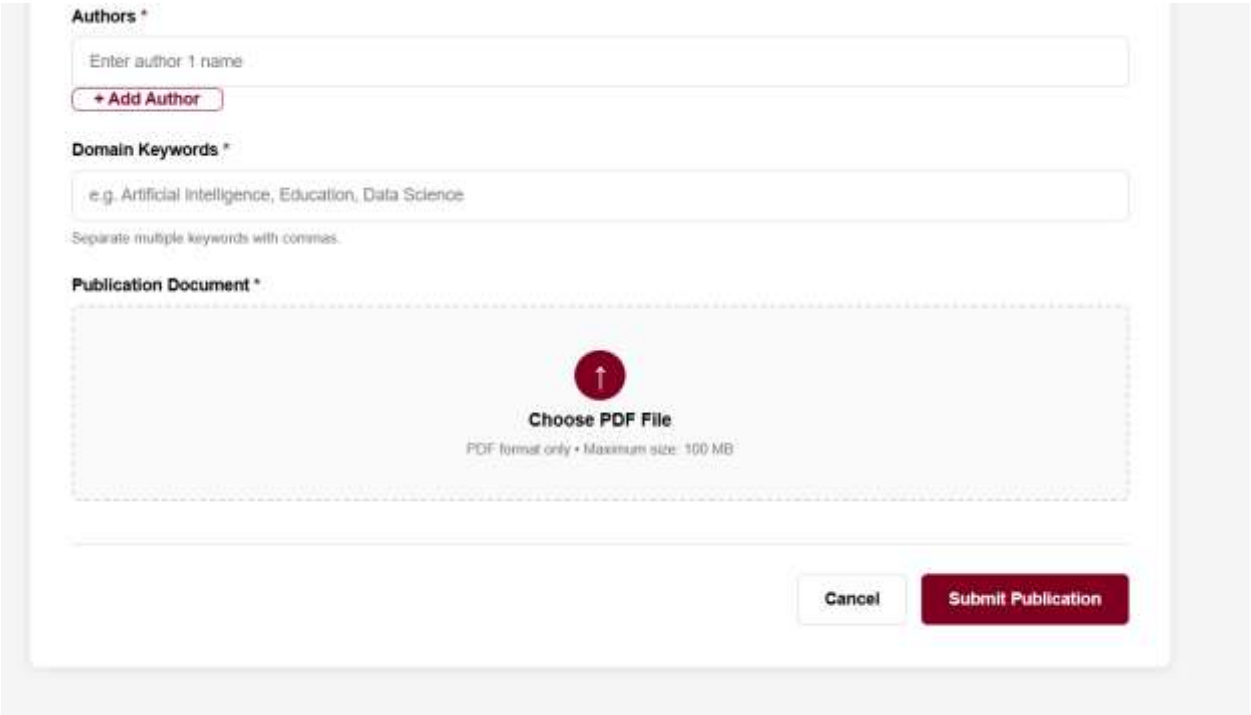


Student dashboard

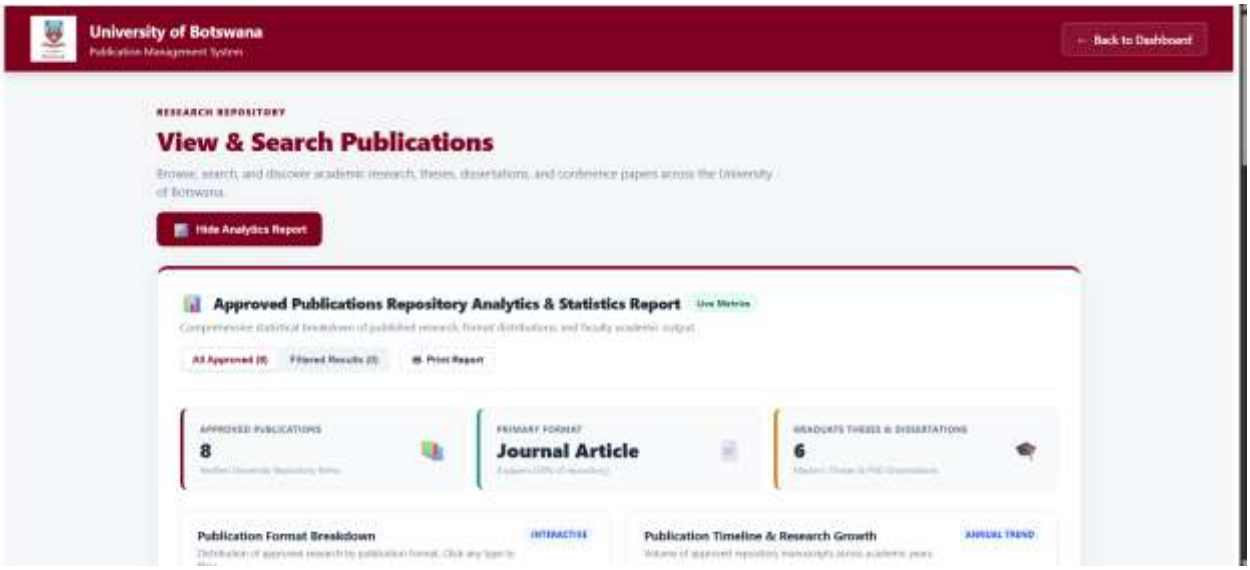


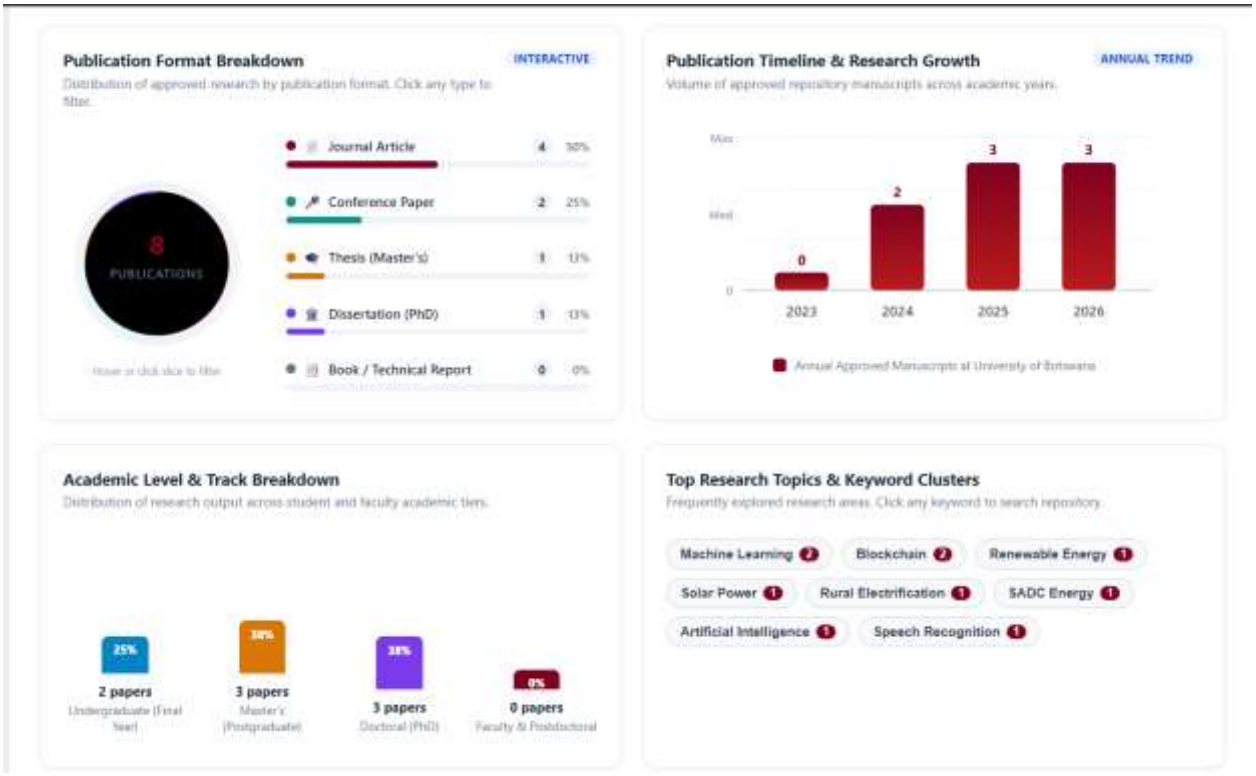
Student submit publication page





View and Search publication page(student)





Detailed Format Summary Registry

8 total records analyzed

Format / Type	Approved Count	Share of Repository	Visual Share	Quick Action
Journal Article	4	50%		Filter View →
Conference Paper	2	25%		Filter View →
Thesis (Master's)	1	13%		Filter View →
Dissertation (PhD)	1	13%		Filter View →
Book / Technical Report	0	0%		Filter View →

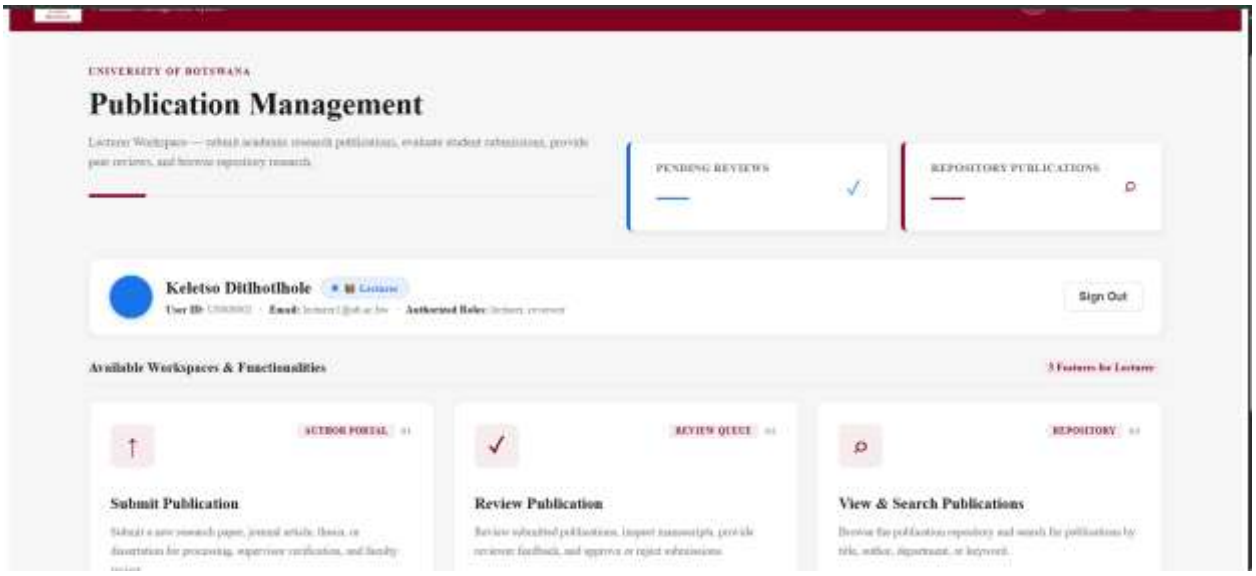
Search publications by title, author, keyword, or reference (e.g. PUB-REF)...

Publication Type: All Types | Year: All Years | Subject Area: All Subjects | Sort By: Newest First | Grid | List

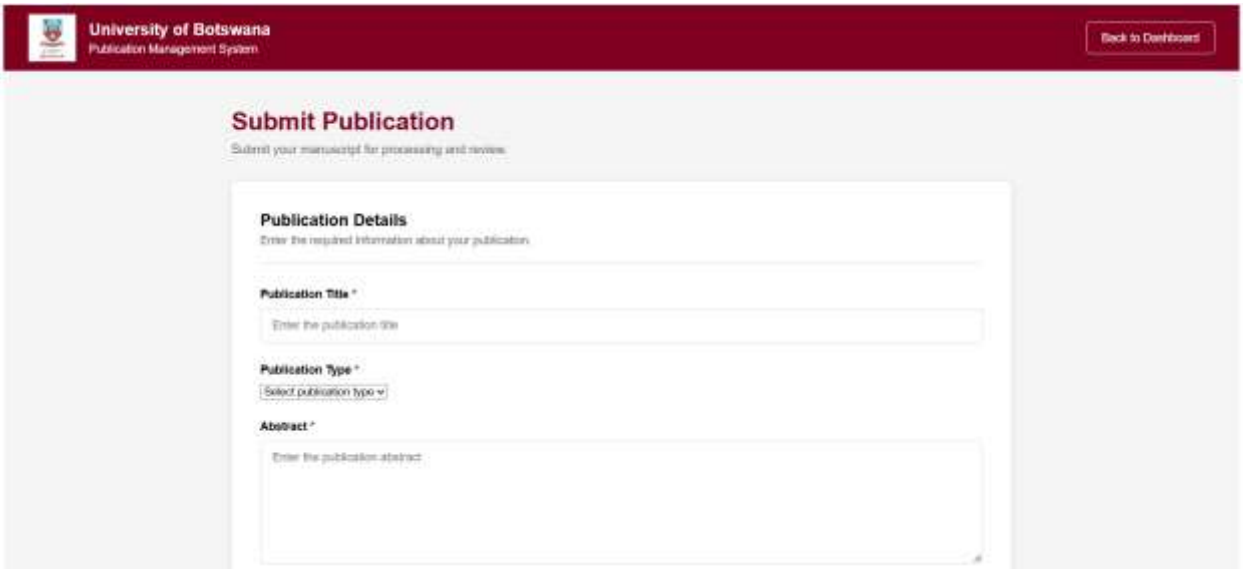
Submitted From: dd/mm/yyyy | To: dd/mm/yyyy

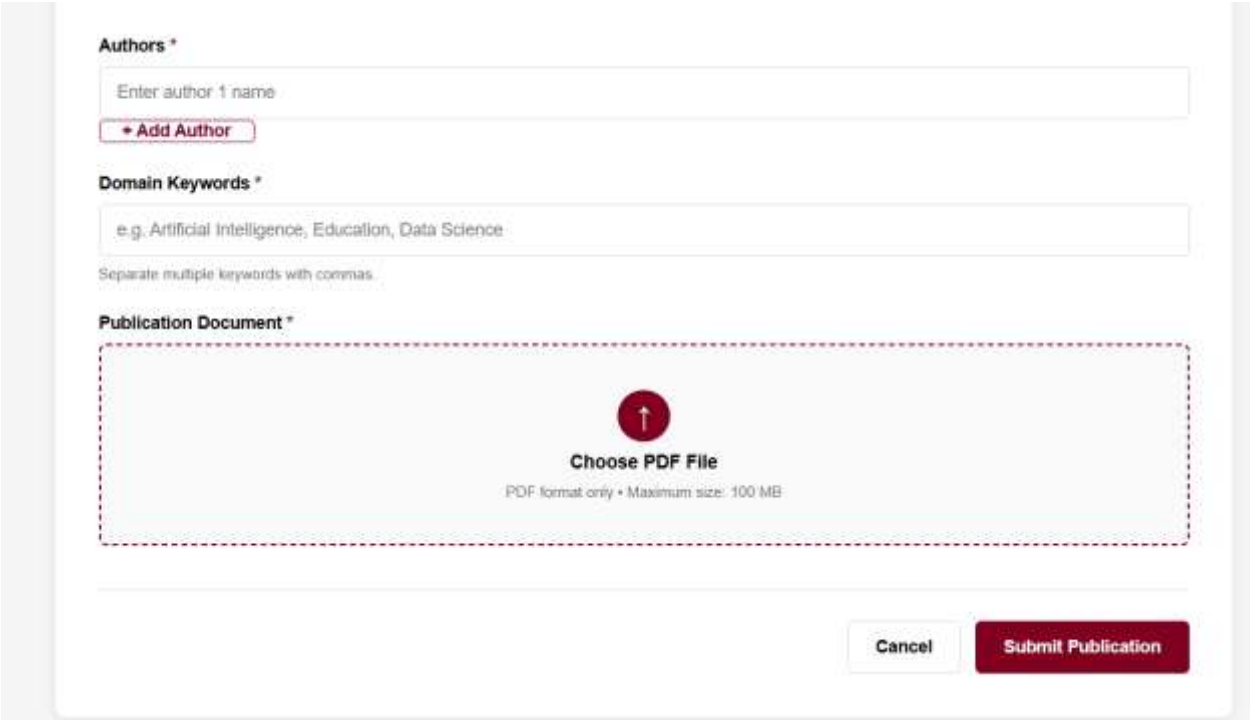
ublications found

Lecturer Dashboard

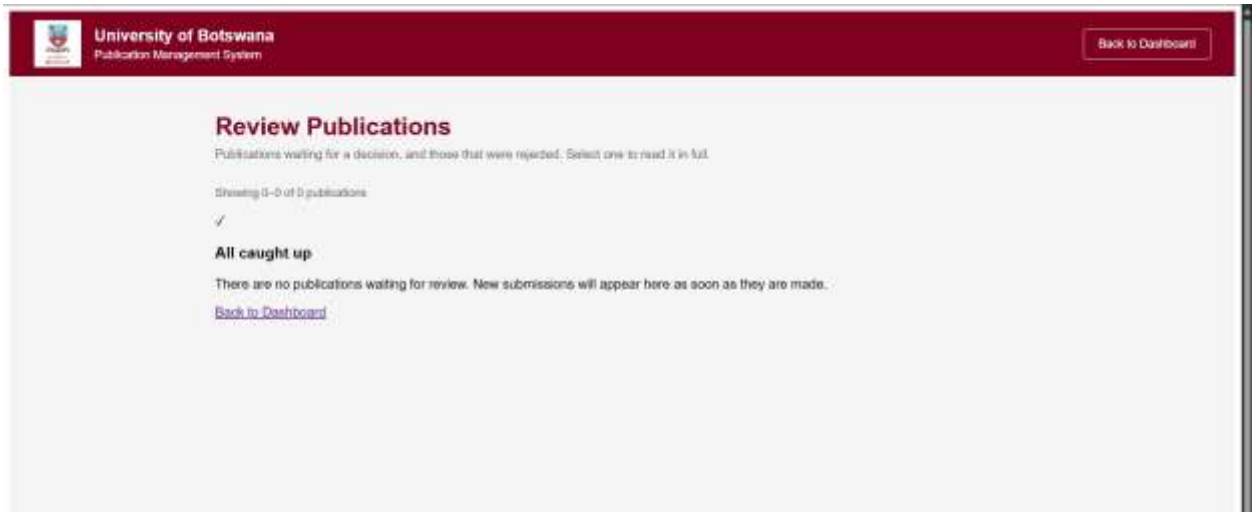


Submit publication-Lecturer

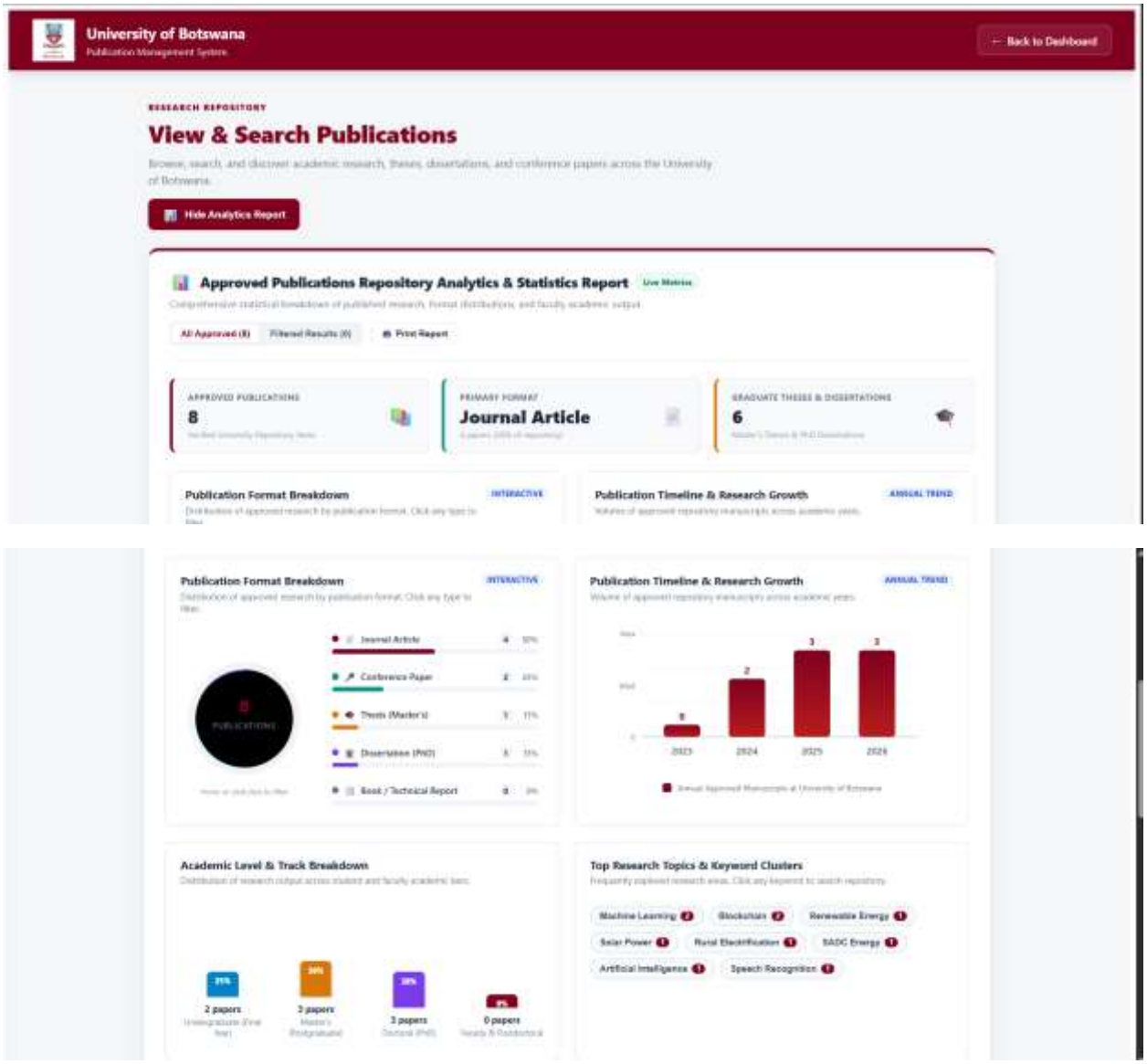


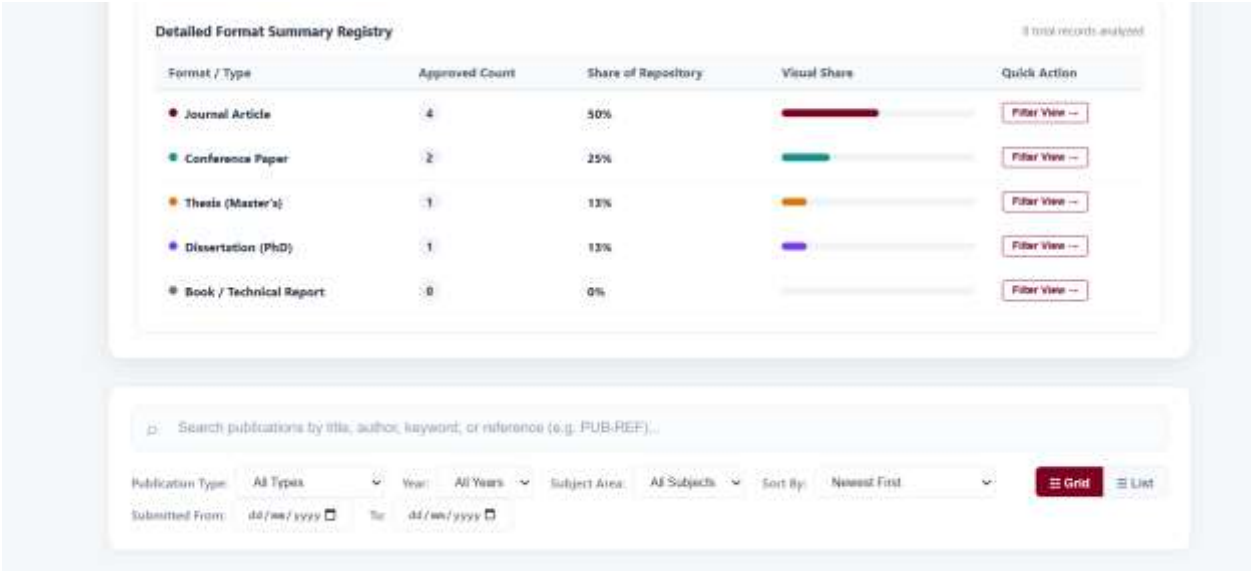


Review Publication Lecture Page

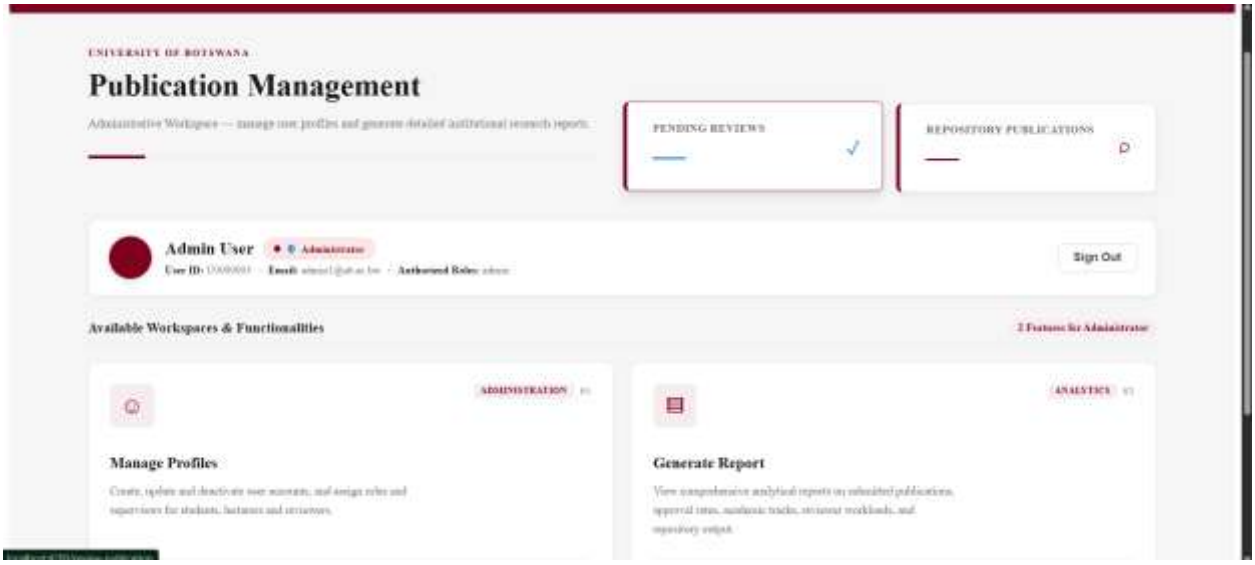


View and Search Lecturer-Page





Admin dashboard



Generate Report

